

可以，下面是 三组伪代码题索引目录版，方便你开卷时快速找题型。📌

伪代码预测题索引目录

1. **MapReduce | WordCount** 关键词: `<word, 1>`、按 word 分组、Reduce 求和。
2. **MapReduce | 倒排索引 Inverted Index** 关键词: `<DocID, content> → <word, list(DocID)>`，Reduce 去重合并。
3. **MapReduce | 反向 Web-Link Graph** 关键词: `<target, source> → <target, list(source)>`。
4. **MapReduce | URL 访问次数统计** 关键词: 日志解析 URL, `<url, 1>`，Reduce 求和。
5. **MapReduce | Distributed Grep / 关键词过滤** 关键词: 包含 keyword 的行输出，可写成 map-only。
6. **MapReduce | 每个用户最大/最小/平均值** 关键词: `<user_id, value>`；平均值用 `(sum, count)`。
7. **HBase | Feed 流写扩散/读扩散** 关键词: `post_content`、`user_inbox`、`user_outbox`、大 V 混合推拉。
8. **Storm | 实时 WordCount 拓扑** 关键词: `Spout → SplitBolt → CountBolt → StoreBolt`，计数用 `fieldsGrouping(word)`。
9. **Flink | URL 5分钟窗口 PV** 关键词: Kafka Source、Event Time、Watermark、`keyBy(url)`、滚动窗口。
10. **Flink | Keyed State 去重/趋势检测** 关键词: `keyBy(user_id/sensor_id)`，`MapState / ValueState`。
11. **MapReduce | 每个用户访问过的不同 URL 数量** 关键词: `<user_id, url>`，Reduce 里 set 去重。
12. **MapReduce | 单词在哪些文档及次数** 关键词: `<word, list(DocID:count)>`，Map 端先文档内局部计数。
13. **MapReduce | Top K 热门 URL** 关键词: 两个 Job；先统计 URL 次数，再局部/全局 Top K。
14. **MapReduce | 按年份统计最高气温** 关键词: `<year, temp>`，Reduce 取 max。
15. **MapReduce | 树结构 parent -> children** 关键词: 输入 `<child, parent>`，输出 `<parent, list(child)>`。
16. **HBase | 读取某用户最近 N 条 Feed** 关键词: `user_inbox`，RowKey=`user_id`，列名=`reverse_timestamp#post_id`。
17. **HBase | 传感器时序数据表设计** 关键词: RowKey=`sensor_id#reverse_timestamp`，时间范围 Scan。
18. **Storm | 实时异常检测拓扑** 关键词: `MetricSpout → ParseBolt → FilterBolt → AlertBolt`，按 `host_id` 用 `fieldsGrouping`。
19. **Flink | 10分钟窗口 UV 去重** 关键词: Event Time、Watermark、Window、SetState 保存 `user_id`。

20. **Flink | Checkpoint 恢复流程** 关键词：恢复 state + source offset, 重启 application, 从最近 checkpoint 继续。
21. **MapReduce | Reduce-side Join** 关键词：两表 join, Map 打 tag, Reduce 按 `user_id` 拼接。
22. **MapReduce | 二次排序**：用户日志按时间排序 关键词：组合 key (`user_id,timestamp`), Partitioner 只按 `user_id`。
23. **MapReduce | 共同好友 / 二度关系推荐** 关键词：好友列表两两组合, `<user_pair, common_friend>`。
24. **MapReduce | 每个地区每小时访问量** 关键词：组合 key (`region,hour`), Reduce 求和。
25. **Spark | RDD 写日志 PV 统计** 关键词：`map -> map -> reduceByKey -> saveAsTextFile`, 标 Transformation / Action。
26. **Spark | 每个用户消费总额并过滤高价值用户** 关键词：`reduceByKey` 求和, `filter` 过滤, `reduceByKey` 产生 Shuffle。
27. **Storm | 实时日志清洗 + URL 热点统计** 关键词：`KafkaSpout -> ParseBolt -> FilterBolt -> CountBolt -> StoreBolt`, 按 URL fieldsGrouping。
28. **Flink | Session Window 统计用户会话点击数** 关键词：用户 30 分钟无事件则关闭 session, `keyBy(user_id)` + Session Window。
29. **Flink | Broadcast State 动态规则告警** 关键词：规则流 broadcast, 指标流 connect, 规则状态广播到所有 task。
30. **Dremel | 字段路径最大 r / d 计算** 关键词：`max_r` 数 repeated, `max_d` 数 repeated + optional, required 不算。

考场快速定位口诀

看到 统计每个 X 的次数：翻 WordCount / URL 统计。看到 每个 X 的最大最小平均：翻用户最大值/天气最高温。看到 去重数量：翻不同 URL 数量 / Flink UV。看到 **Top K**：翻 Top K 热门 URL。看到 连接两张表：翻 Reduce-side Join。看到 按时间排序：翻二次排序。看到 社交关系/树/图：翻 parent-children、共同好友、反向链接图。看到 **HBase 表设计**：翻 Feed 流或传感器时序。看到 **Storm 拓扑**：翻 WordCount、异常检测、日志热点。看到 **Flink 时间窗口**：翻 PV、UV、Session Window。看到 **Flink 状态/恢复**：翻 Keyed State、Broadcast State、Checkpoint。看到 **Dremel r/d**：翻最大 r/d 计算。

1. MapReduce: WordCount

题目预测：给定大量文本文件，统计每个单词出现次数，写出 Map 和 Reduce 伪代码。

```
map(file_id, line):
    for word in split(line):
        emit(word, 1)

reduce(word, counts):
    total = 0
```

```
for c in counts:
    total += c
emit(word, total)
```

答题点：Map 输出 `<word, 1>`，Shuffle 按 `word` 聚合，Reduce 对所有 1 求和。PPT WordCount 明确要求 Map 输入文件 id/文本内容，输出 word/1，Reduce 输入 word 和一连 1，输出 word 和总和。

2. MapReduce：倒排索引 Inverted Index

题目预测：输入 `<DocID, content>`，输出 `<word, list(DocID)>`，写 MapReduce。

```
map(doc_id, content):
    words = split(content)
    for word in words:
        emit(word, doc_id)

reduce(word, doc_ids):
    result = unique(doc_ids)
    emit(word, result)
```

答题点：中间 key 是 `word`，因为最后要按单词聚合。Reduce 里要去重，否则同一个文档里一个词出现多次会重复记录 DocID。

3. MapReduce：反向 Web-Link Graph

题目预测：输入网页链接关系 `<target, source>`，输出每个网页被哪些网页指向：`<target, list(source)>`。

```
map(target, source):
    emit(target, source)

reduce(target, sources):
    result = list(sources)
    emit(target, result)
```

答题点：这个题 Map 几乎不用变换，核心是 Reduce 按 `target` 聚合所有 `source`。PPT 里把 Reverse Web-Link Graph 列为 MapReduce 可表达的典型任务。

4. MapReduce：URL 访问次数统计

题目预测：给定大量访问日志，统计每个 URL 的总访问次数。

```
map(offset, log_line):
    url = extract_url(log_line)
    emit(url, 1)

reduce(url, counts):
    total = 0
    for c in counts:
        total += c
    emit(url, total)
```

答题点：这就是 WordCount 换皮。**word** 换成 **url**。看到“统计每个 X 的次数”，Map 一般输出 **<X,1>**，Reduce 求和。

5. MapReduce：Distributed Grep / 日志关键词过滤

题目预测：给定海量日志，找出包含指定关键词 **keyword** 的所有行。

```
map(file_id, line):
    if contains(line, keyword):
        emit(file_id, line)

reduce(file_id, lines):
    for line in lines:
        emit(file_id, line)
```

更简单写法：这个可以写成 **map-only job**：

```
map(file_id, line):
    if contains(line, keyword):
        emit(file_id, line)
```

答题点：如果只是过滤，不需要聚合，可以没有 Reduce。看到“筛选、过滤、查找包含某词的记录”，优先想到 map-only。

6. MapReduce：求每个用户的最大/最小/平均值

题目预测：给定日志 **<user_id, value>**，求每个用户的最大值、最小值或平均值。

最大值：

```
map(offset, record):
    user_id, value = parse(record)
    emit(user_id, value)
```

```
reduce(user_id, values):
    max_value = -INF
    for v in values:
        if v > max_value:
            max_value = v
    emit(user_id, max_value)
```

平均值:

```
map(offset, record):
    user_id, value = parse(record)
    emit(user_id, (value, 1))

reduce(user_id, pairs):
    sum = 0
    count = 0
    for (v, c) in pairs:
        sum += v
        count += c
    emit(user_id, sum / count)
```

答题点： 平均值不要直接对局部平均值再平均。Combiner 如果要用，应该传 **(sum, count)**，最后再除。

7. HBase: Feed 流写扩散 / 读扩散伪代码

题目预测： 设计社交 Feed 流。普通用户发帖写扩散，大 V 发帖只写 outbox 或推给活跃粉丝。写出伪代码和表结构。

表结构固定写：

```
post_content:
    RowKey = post_id
    CF = meta, content

user_outbox:
    RowKey = author_id
    Column = reverse_timestamp#post_id

user_inbox:
    RowKey = receiver_id
    Column = reverse_timestamp#post_id

follower_list:
    RowKey = author_id
    Column = follower_id

active_follower_list:
```

```
RowKey = vip_author_id
Column = active_follower_id
```

普通用户发帖:

```
publish_normal_user(author_id, post):
    post_id = generate_id()
    put(post_content, post_id, post)

    col = reverse_timestamp(now) + "#" + post_id
    put(user_outbox, author_id, col, post_id)

    followers = scan(follower_list, author_id)
    for follower_id in followers:
        put(user_inbox, follower_id, col, post_id)
```

大 V 发帖:

```
publish_vip_user(author_id, post):
    post_id = generate_id()
    put(post_content, post_id, post)

    col = reverse_timestamp(now) + "#" + post_id
    put(user_outbox, author_id, col, post_id)

    active_followers = scan(active_follower_list, author_id)
    for follower_id in active_followers:
        put(user_inbox, follower_id, col, post_id)
```

读 Feed:

```
read_feed(user_id):
    post_ids = scan_latest(user_inbox, user_id, N)

    vip_followees = scan_vip_followees(user_id)
    for vip_id in vip_followees:
        post_ids += scan_latest(user_outbox, vip_id, K)

    post_ids = sort_by_time(post_ids)
    return batch_get(post_content, topN(post_ids))
```

答题点: HBase 题不是写算法多复杂, 而是写清楚 **RowKey**、**Column**、**访问模式**。HBase 本来就是为高并发随机读写、海量大表设计的; PPT里也强调 HBase 支持随机 realtime read/write, 并能承载 billions rows × millions columns。

8. Storm: 实时 WordCount 拓扑

题目预测：用 Storm 设计实时 WordCount，说明 Spout、Bolt 和 Grouping。

```
SentenceSpout
    emit(sentence)

SplitBolt
    execute(sentence):
        for word in split(sentence):
            emit(word)

CountBolt
    state: count_map

    execute(word):
        count_map[word] += 1
        emit(word, count_map[word])

StoreBolt
    execute(word, count):
        save(word, count)
```

拓扑连接：

```
SentenceSpout --shuffleGrouping--> SplitBolt
SplitBolt --fieldsGrouping(word)--> CountBolt
CountBolt --shuffleGrouping--> StoreBolt
```

答题点：**SplitBolt** 前面用 shuffle，因为拆句子是无状态；进入 **CountBolt** 必须用 **fieldsGrouping(word)**，保证相同 word 到同一个 Bolt 实例。Storm PPT 里明确说 Topology 包含 Spout、Bolt、Stream、Tuple、Grouping，并列出 shuffleGrouping 和 fieldsGrouping。

9. Flink: 按 URL 做 5 分钟窗口 PV 统计

题目预测：给定点击日志流，按事件时间统计每个 URL 每 5 分钟 PV，要求考虑乱序。

```
stream = KafkaSource()

events = stream
    .map(parse_log)
    .filter(event is valid)
    .assignTimestampsAndWatermarks(
        event_time = event.timestamp,
        max_out_of_orderness = 1 minute
    )
```

```
result = events
    .keyBy(event.url)
    .window(TumblingEventTimeWindow(5 minutes))
    .sum(event.count)

result.addSink(database)
```

更像考试的文字版：

```
Source读取Kafka日志
-> map解析日志
-> filter过滤无效数据
-> 提取event_time并生成watermark
-> keyBy(url)
-> 5分钟滚动窗口
-> sum(count)
-> Sink输出
```

答题点： 只要题目出现“事件时间、乱序、窗口”，就一定写 **Event Time + Watermark + Window**。PPT 中 Watermark 定义为 **watermark(t)** 表示时间戳不大于 t 的数据都已到来，可触发和销毁窗口。

10. Flink：基于 Keyed State 的去重 / 状态检测

题目预测： 给定用户行为流，对每个用户去重，或判断某个用户是否重复点击。

去重伪代码：

```
stream = KafkaSource()

result = stream
    .map(parse_event)
    .keyBy(event.user_id)
    .process(DeduplicateFunction)

DeduplicateFunction:
    state seen_ids: MapState<event_id, boolean>

    process(event):
        if seen_ids.contains(event.event_id):
            return
        else:
            seen_ids.put(event.event_id, true)
            emit(event)
```

趋势检测伪代码，比如温度连续上升：

```
stream
    .keyBy(sensor_id)
    .process(RisingTemperatureFunction)

RisingTemperatureFunction:
    state last_temp: ValueState<double>

    process(event):
        old = last_temp.value()
        if old != null and event.temp > old:
            emit("temperature rising")
            last_temp.update(event.temp)
```

答题点：按用户、设备、URL 这种 key 维护历史，一定写 **keyBy + Keyed State**。PPT里说 Keyed State 适用于 key 相关 operator，Flink 为每个 key 维护 state，并保证相同 key 的记录进入维护该 state 的 subtask。

11. MapReduce：统计每个用户访问过的不同 URL 数量

题目预测：输入大量日志，每条记录包含 `user_id`, `url`，统计每个用户访问过多少个不同 URL。

```
map(offset, log_line):
    user_id, url = parse(log_line)
    emit(user_id, url)

reduce(user_id, urls):
    unique_urls = set()
    for url in urls:
        unique_urls.add(url)
    emit(user_id, size(unique_urls))
```

答题点：中间 key 是 `user_id`，因为最后按用户聚合。Reduce 里要 `set` 去重，不是直接 count。

12. MapReduce：每个单词出现在哪些文档及次数

题目预测：倒排索引升级版：输出 `<word, list(DocID:count)>`。

```
map(doc_id, content):
    local_count = map()
    for word in split(content):
        local_count[word] += 1

    for word, count in local_count:
        emit(word, (doc_id, count))

reduce(word, doc_count_pairs):
    result = []
```

```
for (doc_id, count) in doc_count_pairs:
    result.append(doc_id + ":" + count)
emit(word, result)
```

答题点：比普通倒排索引多了“在每个文档内计数”。Map 端可以先本地统计，减少输出量。

13. MapReduce：Top K 热门 URL

题目预测：给定访问日志，求访问次数最高的前 K 个 URL。

这个通常要两个 MR Job。

Job 1：统计每个 URL 次数

```
map(offset, log_line):
    url = extract_url(log_line)
    emit(url, 1)

reduce(url, counts):
    emit(url, sum(counts))
```

Job 2：求 Top K

```
map(url, count):
    emit("topk", (url, count))

reduce("topk", url_counts):
    heap = min_heap(size=K)

    for (url, count) in url_counts:
        heap.add_or_replace_if_larger((url, count))

    emit("TopK", heap.sorted_desc())
```

答题点：第一步是 WordCount 换皮；第二步把所有 `<url, count>` 聚到一起求 Top K。考试如果担心单个 Reduce 压力大，可以写“先局部 Top K，再全局 Top K”。

14. MapReduce：按年份统计最高气温

题目预测：输入天气记录，包含 `year`，`temperature`，输出每年最高气温。

```
map(offset, record):
    year = extract_year(record)
    temp = extract_temperature(record)
    emit(year, temp)
```

```
reduce(year, temps):
    max_temp = -INF
    for t in temps:
        if t > max_temp:
            max_temp = t
    emit(year, max_temp)
```

答题点：看到“每个年份/每个地区/每个用户的最大值”，中间 key 就是年份/地区/用户。Reduce 取 max。

15. MapReduce：树结构，按 parent 聚合 children

题目预测：输入 `<child, parent>`，输出每个节点的孩子列表 `<parent, list(child)>`。

```
map(child, parent):
    emit(parent, child)

reduce(parent, children):
    child_list = []
    for child in children:
        child_list.append(child)
    emit(parent, child_list)
```

答题点：这很像 PPT 里“图/链接关系”类题的变形。核心永远是：最后要按谁聚合，谁就是 key。MapReduce PPT 中也强调了反向 Web-Link Graph、倒排索引等都可以用统一模型表达。

16. HBase：根据时间范围读取某用户最近 N 条 Feed

题目预测：基于 HBase 设计 Feed 表，并写出读取某用户最近 N 条动态的伪代码。

表设计：

```
user_inbox:
    RowKey = user_id
    ColumnFamily = posts
    Column = reverse_timestamp#post_id
    Value = post_id

post_content:
    RowKey = post_id
    ColumnFamily = meta/content
    Value = 正文、作者、时间等
```

读取：

```
read_recent_feed(user_id, N):
    post_ids = []

    columns = scan_columns(
        table = user_inbox,
        rowkey = user_id,
        family = "posts",
        limit = N
    )

    for col in columns:
        post_id = value(col)
        post_ids.append(post_id)

    posts = batch_get(post_content, post_ids)
    return posts
```

答题点: `reverse_timestamp#post_id` 是为了让最新内容排在前面。正文和索引分离, 避免 inbox 里重复存大文本。HBase 适合这种按 RowKey 的随机读写和海量稀疏表场景。

17. HBase: 传感器时序数据表设计与查询

题目预测: 有大量传感器数据 `<sensor_id, timestamp, value>`, 设计 HBase 表并写查询某传感器某时间段数据的伪代码。

表设计:

```
sensor_data:
    RowKey = sensor_id#reverse_timestamp
    ColumnFamily = data
    Columns:
        data:value
        data:type
        data:status
```

写入:

```
write_sensor(sensor_id, timestamp, value):
    rowkey = sensor_id + "#" + reverse_timestamp(timestamp)
    put(sensor_data, rowkey, "data:value", value)
```

读取某时间段:

```
query_sensor(sensor_id, start_time, end_time):
    start_row = sensor_id + "#" + reverse_timestamp(end_time)
```

```
end_row    = sensor_id + "#" + reverse_timestamp(start_time)

rows = scan(sensor_data, start_row, end_row)

result = []
for row in rows:
    result.append(row["data:value"])

return result
```

答题点：HBase 表设计永远围绕访问模式。这里主要按 **sensor_id + 时间范围** 查，所以 RowKey 要把 sensor_id 放前面，时间放后面。

18. Storm：实时异常检测拓扑

题目预测：给定服务器指标流，实时检测 CPU 或 QPS 是否超过阈值并报警。

```
MetricSpout:
    emit(metric)

ParseBolt:
    execute(metric):
        host_id, metric_name, value, timestamp = parse(metric)
        emit(host_id, metric_name, value, timestamp)

FilterBolt:
    execute(host_id, metric_name, value, timestamp):
        if metric_name in ["cpu", "qps", "memory"]:
            emit(host_id, metric_name, value, timestamp)

AlertBolt:
    state: last_values

    execute(host_id, metric_name, value, timestamp):
        if value > threshold(metric_name):
            emit_alert(host_id, metric_name, value, timestamp)
```

拓扑连接：

```
MetricSpout --shuffleGrouping--> ParseBolt
ParseBolt --shuffleGrouping--> FilterBolt
FilterBolt --fieldsGrouping(host_id)--> AlertBolt
```

答题点：解析、过滤是无状态，用 shuffle；如果报警要按机器维护历史状态，就用 **fieldsGrouping(host_id)**。Storm 的计算模型就是 Spout 产生 tuple，Bolt 处理 tuple，Grouping 决定路由。

19. Flink：每个用户 10 分钟内去重 UV

题目预测： 给定点击流，统计每个 URL 每 10 分钟窗口内的 UV，考虑事件时间乱序。

```
stream = KafkaSource()

events = stream
    .map(parse_log)
    .assignTimestampsAndWatermarks(
        event_time = event.timestamp,
        max_out_of_orderness = 1 minute
    )

uv = events
    .keyBy(event.url)
    .window(TumblingEventTimeWindow(10 minutes))
    .process(CountUVFunction)

CountUVFunction:
    state seen_users: SetState<user_id>

    process(event):
        seen_users.add(event.user_id)

    on_window_close():
        emit(url, size(seen_users))
        clear(seen_users)
```

答题点： PV 是 count，UV 是去重，所以要保存 **seen_users** 状态。事件时间乱序就写 Watermark。Flink PPT 里重点就是事件时间、水印、有状态计算和窗口。

20. Flink：Checkpoint 恢复流程伪代码

题目预测： 给一个有状态程序，说明发生故障后如何从 checkpoint 恢复。

```
normal_run():
    while job is running:
        process_records()
        update_operator_state()

        if checkpoint_barrier_arrives:
            snapshot_state_to_remote_storage()
            save_source_offset()
            mark_checkpoint_complete()

    on_failure():
        restart_application()

    checkpoint = load_latest_completed_checkpoint()
```

```
for each stateful_subtask:
    restore_state(subtask, checkpoint.state[subtask])

for each source:
    restore_offset(source, checkpoint.offset[source])

resume_processing()
```

答题点：要写两个东西一起恢复：**状态 state** 和 **输入位置 offset**。只恢复状态不恢复 Kafka offset，会重复或漏处理。PPT里说 Flink 从 checkpoint 恢复时会重启 application，把 stateful subtasks 恢复到最近 checkpoint，并配合 Kafka 实现 exactly-once。

21. MapReduce：Reduce-side Join，两张表连接

题目预测：有用户表 `<user_id, user_name>` 和订单表 `<user_id, order_id, amount>`，输出每个订单对应的用户名。

```
map(record_id, record):
    if record comes from user_table:
        user_id, user_name = parse_user(record)
        emit(user_id, ("user", user_name))

    if record comes from order_table:
        user_id, order_id, amount = parse_order(record)
        emit(user_id, ("order", order_id, amount))

reduce(user_id, values):
    user_name = null
    orders = []

    for v in values:
        if v.type == "user":
            user_name = v.user_name
        else if v.type == "order":
            orders.append(v)

    for order in orders:
        emit(order.order_id, (user_id, user_name, order.amount))
```

答题点：两张表 join，Map 端给不同来源打 tag，Reduce 按 `user_id` 聚合后拼接。

22. MapReduce：按用户输出访问日志，并按时间排序

题目预测：输入日志 `<user_id, timestamp, url>`，输出每个用户按时间排序的访问序列。

```
map(offset, log_line):
    user_id, timestamp, url = parse(log_line)
    emit((user_id, timestamp), url)

partitioner(key):
    user_id, timestamp = key
    return hash(user_id) mod R

sort_key(key):
    return (user_id, timestamp)

reduce((user_id, timestamp), urls):
    for url in urls:
        emit(user_id, (timestamp, url))
```

更好理解的考试解释：中间 key 用 `(user_id, timestamp)`，排序时先按 `user_id`，再按 `timestamp`；Partitioner 只按 `user_id` 分区，保证同一个用户到同一个 Reduce。

答题点：这是“二次排序”题。重点不是代码，而是写出：分区按 `user_id`，排序按 `user_id+timestamp`。

23. MapReduce：共同好友 / 二度关系推荐

题目预测：输入每个用户的好友列表，输出两个用户的共同好友数量，用于好友推荐。

输入：

```
A: B C D
B: A C
C: A B D
```

Map：

```
map(user, friends):
    friend_list = parse(friends)

    for each pair (f1, f2) in combinations(friend_list, 2):
        key = sort_pair(f1, f2)
        emit(key, user)
```

Reduce：

```
reduce(user_pair, mutual_friends):
    count = 0
    list = []
```

```
for friend in mutual_friends:
    count += 1
    list.append(friend)

emit(user_pair, (count, list))
```

答题点：对于用户 A 的好友 B、C、D，说明 B-C、B-D、C-D 都有共同好友 A。中间 key 是“用户对”。

24. MapReduce：统计每个地区每小时访问量

题目预测：输入日志 `<ip, timestamp, url>`，根据 IP 得到地区，统计每个地区每小时 PV。

```
map(offset, log_line):
    ip, timestamp, url = parse(log_line)
    region = ip_to_region(ip)
    hour = extract_hour(timestamp)
    emit((region, hour), 1)

reduce((region, hour), counts):
    total = 0
    for c in counts:
        total += c
    emit((region, hour), total)
```

答题点：看到“每个A每个B”，中间 key 就写成组合 key：(A, B)。这里就是 (region, hour)。

25. Spark：用 RDD 写日志 PV 统计

题目预测：用 Spark RDD 统计每个 URL 的访问次数，并说明哪些是 Transformation，哪个是 Action。

```
val logs = sc.textFile("hdfs://logs")

val urlCounts = logs
    .map(line => extractUrl(line))
    .map(url => (url, 1))
    .reduceByKey((a, b) => a + b)

urlCounts.saveAsTextFile("hdfs://output")
```

答题点：textFile、map、reduceByKey 都形成 RDD 转换关系；saveAsTextFile 是 Action，触发执行。PPT里 Spark WordCount 就强调 flatMap、map、reduceByKey 是 Transformation，saveAsTextFile 是 Action。

26. Spark：求每个用户消费总额并过滤高价值用户

题目预测： 输入订单日志 `<user_id, amount>`，求消费总额大于 10000 的用户。

```
val orders = sc.textFile("hdfs://orders")

val result = orders
  .map(line => {
    val user_id, amount = parseOrder(line)
    (user_id, amount)
  })
  .reduceByKey((a, b) => a + b)
  .filter(pair => pair._2 > 10000)

result.saveAsTextFile("hdfs://vip_users")
```

答题点： `reduceByKey` 会产生 Shuffle，所以如果问 Stage，通常在 `reduceByKey` 前后切开。`filter` 是窄依赖，可以和后续写出阶段连在一起。

27. Storm：实时日志清洗 + 按 URL 统计热点

题目预测： 设计 Storm 拓扑，实时解析访问日志、过滤无效日志、统计 URL 热点。

```
KafkaSpout:
  emit(raw_log)

ParseBolt:
  execute(raw_log):
    user_id, url, timestamp, status = parse(raw_log)
    emit(user_id, url, timestamp, status)

FilterBolt:
  execute(user_id, url, timestamp, status):
    if status == 200 and url is not null:
      emit(url, 1)

CountBolt:
  state: count_map

  execute(url, one):
    count_map[url] += one
    emit(url, count_map[url])

StoreBolt:
  execute(url, count):
    save_to_db(url, count)
```

拓扑连接：

```
KafkaSpout --shuffleGrouping--> ParseBolt
ParseBolt --shuffleGrouping--> FilterBolt
FilterBolt --fieldsGrouping(url)--> CountBolt
CountBolt --shuffleGrouping--> StoreBolt
```

答题点：解析和过滤是无状态，用 shuffle；按 URL 统计必须 `fieldsGrouping(url)`。Storm 题核心就是 Spout、Bolt、Tuple、Grouping。

28. Flink：Session Window 统计用户会话长度

题目预测：给定用户点击流，若用户 30 分钟内没有新事件则认为一个 session 结束，统计每个用户每个 session 的点击数。

```
events = KafkaSource()
    .map(parse_click)
    .assignTimestampsAndWatermarks(
        event_time = event.timestamp,
        max_out_of_orderness = 1 minute
    )

sessions = events
    .keyBy(event.user_id)
    .window(EventTimeSessionWindow(gap = 30 minutes))
    .aggregate(CountClicks)

CountClicks:
    add(event, acc):
        acc.count += 1
        return acc

    getResult(acc):
        return acc.count

sessions.addSink(database)
```

答题点：看到“用户一段时间不活跃就结束”，就是 **Session Window**。如果题目提乱序，就加 **Event Time + Watermark**。

29. Flink：Broadcast State 动态规则告警

题目预测：一条流是指标数据，另一条流是动态告警规则。要求实时用最新规则判断是否报警。

```
metrics = KafkaSource("metrics").map(parse_metric)
rules = KafkaSource("rules").map(parse_rule)
```

```
broadcast_rules = rules.broadcast(rule_state_descriptor)

alerts = metrics
    .connect(broadcast_rules)
    .process(AlertFunction)

AlertFunction:
    broadcast_state rules

    processBroadcastElement(rule):
        rules.put(rule.metric_name, rule.threshold)

    processElement(metric):
        threshold = rules.get(metric.metric_name)
        if threshold != null and metric.value > threshold:
            emit_alert(metric.host_id, metric.metric_name, metric.value)
```

答题点：这个题考 **Broadcast State**。规则要广播到所有并行实例；指标流按规则判断。Flink PPT 里 Operator State 包括 Broadcast State，适合每个 task 状态相同的情况。

30. Dremel：给字段路径计算最大 r / 最大 d

题目预测：给定字段路径，判断该列最大 repetition level 和 definition level。

规则伪代码：

```
compute_max_r_d(path):
    max_r = 0
    max_d = 0

    for field in path:
        if field is repeated:
            max_r += 1
            max_d += 1

        else if field is optional:
            max_d += 1

        else if field is required:
            continue

    return (max_r, max_d)
```

例子1：

```
Name repeated
Language repeated
Code required
```

```
max_r = 2  
max_d = 2
```

例子2:

```
Name repeated  
Language repeated  
Country optional  
  
max_r = 2  
max_d = 3
```

答题点： **r** 只数 repeated； **d** 数 repeated + optional；required 不增加 d。Dremel PPT里明确说 r 是 repetition level, d 是 definition level, 并用 **Name.Language.Code** 和 **Name.Language.Country** 做重点例子。
